

02-lab-work-speaking-script

Speaking Script: S3 Code Review and Lab Work

For the one-to-one with Prof. Naik. Internal working document, not a hand-in. It walks the presentation PDF sheet by sheet: what is on the page, what it means, and what to say.

The document you present	<code>group 5 - s3 code review & lab work.pdf</code> , 13 sheets
The backing document	<code>detailed non-functional-requirements.pdf</code> , 8 sheets
Format	One-to-one, screen shared, terminal open alongside
Target	12 minutes spoken, leaving room for questions

Before you start

Have these open, in this tab order. Nothing else.

1. The presentation PDF, on sheet 1
2. A terminal at the repository root, cleared
3. GitHub, on a pull request with the checks visible and green
4. The live site, already loaded twice so it is warm

Two rules for the whole session.

Every claim is followed by the thing that proves it, on screen. Never say "the code quality is good"; say what is enforced and let the build be the evidence.

He has said the whole codebase does not need walking. So **run the tools live rather than describing them**. The terminal is the point of this session.

Bold sentences below are the ones that carry the argument. If you are cut short, say only those.

Sheet 1 and 2: cover and contents (40 seconds)

Do not read the cover out. Give the project in two sentences while it is on screen, then move to the contents page.

Revix is an evidence-traceable review system for Indian cars and two-wheelers. The problem is that a star average hides who rated, what they cared about, and whether they agreed, so **we rebuild the verdict from the reviews underneath it and keep every number traceable to the sentences it came from.**

It runs on real data: **twenty thousand review units across three sources, 143 variants, 42 models**, refreshed by a pipeline that runs itself.

On the contents page, point at the five sections and say you will spend most of the time on section three, the measured code quality, since that is what the session is for.

Do not demo the product. That is the 16 October submission. If he starts clicking, let him, then steer back: "I can show the full flow properly in the demo session; today I would rather show you what is underneath it."

Sheets 3 and 4: codebase (2 minutes)

Sheet 3 is the shape and the scale. **Sheet 4** is the part worth talking about.

On sheet 3, do not read the tree. Say what it is and move:

Three Python packages and a Next.js app, about seventeen thousand lines. The pipeline is the biggest piece and the API is deliberately the smallest, under two thousand lines, because almost nothing happens there.

Then the numbers underneath it:

Twenty thousand evidence units, 143 variants, and **134 of them publish a verdict while nine are held back.** Those nine are not missing data, they are below our evidence floor, and we would rather say nothing than publish a score built on four reviews.

Sheet 4 is where you slow down.

The one rule is that dependencies run in one direction. The API and the pipeline both depend on core, and core depends on neither, so the read path and the write path cannot tangle.

And it is enforced by import linting in CI, not by us remembering. That is also why it is three packages rather than one: in a single package this would be a naming convention, and naming conventions lose. Here the API physically cannot import the pipeline.

Then the schemas, briefly:

The database does the same thing: four schemas separated by lifecycle rather than by subject, so you can always tell which layer a table is in and what is allowed to write to it.

If he asks about the migration checks, the third one is the interesting one: CI checks migrations apply, that they match the models, **and that they reverse cleanly**, which most projects never test.

Sheets 5 and 6: frameworks (2.5 minutes)

Sheet 5 is the table. Do not read it. Let him scan it and give the shape:

Rather than justify these in prose, every choice is written down as a decision record, with the alternative we rejected. Nine of them.

Then point at the closing note on that sheet, which is the actual argument:

Each of these was chosen so that a mistake becomes a failed build rather than a defect a reader finds. The generated client, the typed ORM, the migration checks and the contract-first schema are all the same move.

Sheet 6 has the two worth arguing about. These are your strongest minutes in the first half.

The orchestration one:

We use scheduled GitHub Actions instead of Airflow or Prefect. An orchestrator is a server to run, secure and pay for, and our pipeline is one nightly batch with no fan-out and no backfill. Actions already holds our secrets, already schedules, already alerts.

Choosing Airflow here would have been choosing a tool because it looks professional rather than because the problem needs it.

The machine learning one:

We built a distant-supervision aspect classifier: lexicon labels train a TF-IDF into a one-vs-rest logistic regression.

It lost to the lexicon it learned from by 0.29 macro F1, so it ships disabled, and the number is published on our public metrics page rather than quietly dropped.

The alternative was shipping a model because we had built one, and the whole argument of this project is that a number should be earned.

Do not rush that and do not apologise for it. A negative result you published is a stronger claim than a positive one you did not check.

Sheets 7, 8 and 9: code quality (4 minutes, the core)

This is what the session is for. **Move to the terminal for sheet 7.**

Sheet 7: run the score live

Type it. Do not paste a screenshot. It takes about ten seconds and a number produced in front of him is worth more than a number in a document.

```
uv run radon mi packages/revix_core/src pipeline/src apps/api/src -s
```

This is the maintainability index, the closest thing Python has to a single score for how hard a file will be to change in six months. Zero to a hundred, and anything above twenty is grade A.

Every one of our forty-three files is an A.

Then complexity:

```
uv run radon cc packages/revix_core/src pipeline/src apps/api/src -a -s -nC
```

Cyclomatic complexity counts the independent paths through a block, which is also the minimum number of tests needed to cover it. **The average across 344 blocks is A, at 3.9**, and four fifths of the codebase is in the simplest band. What is listed here is only the tail, anything above B.

Back to sheet 7 and point at the distribution bar, then the toolkit table: Radon scores, Xenon enforces, Ruff and mypy cover lint and types, PyTest runs the suite.

Sheet 8: what the measurement changed

This is the strongest single item in the whole session.

The measurement did not just produce a number, it changed the code. One function graded **E at 33**, the only E in the codebase and forty percent worse than the next worst.

When I read it, it was doing two unrelated jobs: drawing samples from the corpus, and turning those observations into summary statistics. Neither half is complicated. Doing both in one place was.

Split into five functions along that seam, and **the codebase now has no E-grade block at all**. The thirteen tests covering it passed unchanged the whole way through, which is the only reason that refactor was safe to make.

Then volunteer the two at D, before he finds them:

Two blocks still grade D and I would rather show you them than have you find them. `fuse_variant` is the scoring engine and `run_connector` is the ingest loop that holds three failure modes open so one dead source never stops the others.

In both, the branching is the requirement rather than a style accident. Splitting them into functions that only ever run in sequence would move the complexity into the call graph rather than remove it, and both are on the write path, where a refactor that changes a verdict is worse than a D grade.

Sheet 9: enforcing it, and the tool we skipped

Show the Complexity gate step in CI.

Radon will tell you a function scores E and then let you merge it. That is a report, not a control. So we added **Xenon, which is the same measurement wired to an exit code**, and it runs on every pull request.

The thresholds are set where the code actually is, so **a change that makes any of these grades worse does not merge**. That is the difference between a measurement and a property, and it is the part of this I would defend hardest.

Then the omission, before he notices it:

Mutation testing was the most interesting tool on your list and we left it out deliberately. **Mutmut has no native Windows support, and all three of us develop on Windows**. A quality tool that only one machine can run is a tool that stops being run.

The question it answers, whether the tests would actually catch a bug, we answer with evidence instead.

Then the defects table on the same sheet. **Pick two, do not read all five**. The macro-F1 one and the connector one are the best:

Our macro-F1 was averaging over aspects with zero support, so a perfect system scored 0.22 on a two-topic set. **Our evaluation was flattering us**, and a test caught it.

And a connector was recording our own search term as though the source had said it, which means **we would have been citing ourselves as evidence**.

Sheets 10, 11 and 12: non-functional requirements (3 minutes)

Sheet 10: lead with the architecture, not the list

We audited all nine categories against the actual code rather than against what our proposal claimed. **The first thing to say is that six of the nine were already satisfied by one architectural decision**, and I would rather explain that decision than read you a list.

Point at the diagram.

The write path and the read path are strictly separated. Fusion, scoring, confidence intervals and claim generation all happen in the nightly pipeline and land as finished rows. **Every endpoint is one indexed read of a row that already exists. No model runs during a user request.**

That gives us four requirements at once. Performance, because nothing on the read path grows with the corpus, since the corpus is not on the read path. Scalability, because ten times the data changes how long the night takes and changes nothing about a request. Reliability, because a page cannot fail when a third-party model is down. And availability, because a demo depends on nothing remote.

Pause here. This is the sentence the section rests on. Then the two figures underneath: 22 to 35 milliseconds warm, 96 to 247 from India, against a 300 millisecond target.

Sheet 11: the nine, quickly

Do not read the grid. Let him scan it and name the shape:

Nine categories, eight met and one stated honestly. The audit found five real gaps and closed all five: no security headers anywhere, unbounded database waits, no rate limiting, no compression or caching, and no request identity.

Sheet 12: the finding to lead with

If only one NFR point lands, make it this one.

The gap I would most like to tell you about was not a missing feature. It was a requirement we had already implemented, and tested, and still had not met.

Our health endpoint is deliberately written to report an unreachable database as a 503 rather than a stack trace. It was written that way on purpose. There is a test for it.

It could not actually do it. There was no connect timeout, so the driver sat on the operating system's TCP timeout while the hosting platform's probe gave up first. **A correct 503 became no answer at all**, and a hang is worse than an error because nothing downstream can react to it.

What led us there was our own test run hanging for four hundred seconds against a machine with no database. Every wait is now bounded.

The lesson is the difference between writing a non-functional requirement and verifying one. We had the code, the intent, and the test, and the requirement was still not met.

Then close with the two limits, before he finds them.

Two things we state rather than hide. The rate limiter is per instance and in memory, so it is not a global quota. And a single free-tier instance is resilient to its dependencies failing, not highly available; there is no redundancy and no failover.

Both are the first things that would need replacing if this grew.

Sheet 13: close (30 seconds)

Every figure in the document is reproducible in under a minute, and the commands are on this last page. The detailed non-functional requirements document goes through all nine categories properly if you want the working. The site is live now if you want to look at it, and the working demo is the next submission.

Questions to expect

"Why is the classifier disabled? Isn't the ML the point?"

The point is that a number should be earned. We measured it against the lexicon it was trained from and it lost by 0.29 macro F1. Shipping it anyway would contradict the whole argument of the project. The code, the result and the reason are all kept, and it is published on our metrics page.

"How do you know the tests are worth anything?"

The best question you can get. Give the defects from sheet 9, then:

And that is also why we do not gate on coverage. A coverage threshold is trivially satisfied by tests that execute code without asserting anything about it, and we would rather the number stayed honest than stayed high.

"Why not Airflow?"

One nightly batch, no fan-out, no backfill. Actions already holds our secrets, schedules, and alerts. ADR 0002 records the decision and the trigger for revisiting it.

"Your p95 target is 300ms. Have you measured it?"

Yes, and we made it measurable from outside: every response carries an `X-Response-Time-ms` header. Warm and local it is 22 to 35 milliseconds; from India against the deployed instance, 96 to 247. It holds because of the architecture, not tuning: there is nothing on the read path to be slow.

"Only three sources?"

Three that permit us. Team-BHP returns 403 to our agent and two other sites disallow us outright in robots.txt. **We chose to respect that rather than spoof a browser.** Working around it would mean misrepresenting who we are.

"Is any of the data fabricated?"

None of it. Twenty thousand real review units. Where a number does not exist we say so: nine variants sit below our evidence floor and are held back rather than shown.

"Can I see the code?"

Please. Which part? If he has no preference, open `admin.py` for the authentication and the fail-closed rule, or `fuse.py` for the scoring engine, which is the D-grade function from sheet 8.

If something goes wrong

Problem	What to do
A command errors in front of him	Read the error out and diagnose it. A calm diagnosis reads far better than a flustered excuse
CI is red on the day	Open the failing job and read it. A red build you can explain in ten seconds beats a green one you cannot
The live site is cold	"That is the free-tier cold start I mentioned." Turn it into the availability point
He asks for something not built	"Not built, and here is what we chose instead and why." Never invent a feature
You lose the thread	Go back to the five section headings: codebase, frameworks, code quality, non-functional requirements, run it live

Never say "it usually works." Either show it, or say plainly that it is broken and what you would check first.